

Kuopio, Finland
University of Eastern Finland
Aerosol Physics Research Group

Documentation of GCMforcing Analysis Tool

Atte Laakso

Date: August 12, 2025

Glossary and Acronyms

<i>AOD</i>	aerosol optical depth
<i>CDNC</i>	cloud droplet number concentration
<i>CRE</i>	cloud radiative effect
<i>ADRE</i>	aerosol direct radiative effect (also marked as simply <i>DRE</i>)
<i>FI</i>	feature importance
R^2	coefficient of determination
<i>RMSE</i>	root mean squared error

Abstract

This documentation is made for understanding the GCMforcing analysis tool created for studying uncertainties of aerosol direct and indirect forcings. The tool is publicly available in GitHub repository via <https://github.com/TUT-ISI/GCMforcing>. The analysis tool consists of various files containing Python programs, parameters, and tables. The analysis tool can be described to consist of two main parts; data generation and feature analysis.

The files of this analysis system are generally commented and creators and editors of each file are commonly specified. This document is mainly made for clarifying only certain, most crucial aspects of the tool. The entire workflow is now completed for studying feature importances on aerosol direct radiative forcing but it can be modified to fit requirements for other variables too. Examples given in this document along with the comments within the system should guide the reader to using the system correctly. Reading the general description of a certain part of the system is advised before starting testings on that part.

Current iteration documented on 11.08.2025 for the v1.0.0 version of this tool uses data from AeroCom Phase III CONTROL experiment that is accessible via AeroCom server with these instructions: <https://aerocom.met.no/data/download>. The files used as baseline for calculations are from study by Irfan et al. (2023) (<https://doi.org/10.5194/acp-24-8489-2024>). Note that these baseline files are currently only accessible by contacting the author. Public availability will be granted in following updates of this tool. See all the requirements for operating this tool in section 2.

For answers to more in detail questions, please contact the person marked as the editor/creator of the program in question.

Contents

1	Introduction	5
2	Mandatory Requirements for Operation	6
3	Naming the Files	8
4	Data Generation Tools	9
4.1	Aerosol Direct Radiative Effect Calculation	10
4.1.1	DRE_runner.py	10
4.1.2	ADRE calculation and DRE_offline_driver.py	10
4.2	CDNC and CRE calculation	11
4.2.1	CDNC_runner.py	12
4.2.2	CDNC_offline_driver.py	12
4.3	Cautions on the CDNC Pipeline and Development Suggestions	12
5	Feature Importance Analysis Tools	13
5.1	In General	13
5.2	The Initialization of FI Analysis	13
5.3	The Domain-wise ML Development	14
5.4	The All Data ML Development	14
5.5	Other Files in FI Analysis Tools	15
5.5.1	additional.py	15
5.5.2	ML_additional.py	15
6	Step-by-Step Instructions	15
6.1	Calculating ADRE, AOD, Absorption and Average Backscatter Fraction Values on Compute Nodes	15
6.2	Calculating $\Delta ADRE$	16
6.3	Calculating CDNC and (aerosol-cloud interaction based) Energy Balance on Compute Nodes	16
6.4	Initializing data for FI analysis using get_data.py	17
6.5	Calculating Feature Importances on Domain-wise ML Models	17
6.6	Calculating Feature Importances on All Data ML Models . .	18
6.7	Adding New Aerosol Models to the System	19
6.8	Adding New Features to the System	19
6.9	Adding New ML Models	20
7	More Information	21

1 Introduction

The purpose of the whole analysis tool is to study inter-model uncertainty in aerosol direct and indirect effect. The tool calculates offline different target datasets (basically aerosol direct/indirect effects) based on simulated properties of ECHAM6.0-SALSA2.0 model while interchanging data for one or more parameters from another aerosol model. Therefore, the results of these target calculations, where we are interchanging data from different models, are only different from each other because of the differences in the original models as all models have the same initialization when parameter data is formed. Hence, it is possible to determine which parameter causes the most inter-model variability using then the feature importance analysis part of this tool by forming the feature importance values with different machine learning models.

The data generating part of this analysis tool consists of two main parts; offline driver and runner. They include the most essential parts for operating the data generation. Offline driver is made for running different smaller scripts and for calculating correct output for corresponding files according to the instruction it gets from the runner. Offline driver uses model data and different look-up files in order to calculate needed properties for calculating aerosol optical depth (*AOD*), cloud droplet number concentration (*CDNC*), cloud radiative effect (*CRE*), aerosol direct radiative effect (*ADRE* or simply *DRE*), absorption and extinction levels. Note that all these calculations above are not in the same workflow but rather there are two distinct pipelines; one for *AOD* and direct effect calculations and one for *CDNC* and indirect effect calculations.

Subsequently, there are currently two versions of offline drivers; `DRE_offline_driver.py` for calculating *ADRE* and `CDNC_offline_driver.py` for *CDNC* and *CRE*. Runner files for these different offline drivers are `DRE_runner.py` for the `DRE_offline_driver.py` and `CDNC_runner.py` for the `CDNC_offline_driver.py`. Runner files are used for calculating multiple outcomes without having to manually insert all the parameters in between the runs. Runner files are run on compute nodes to speed up the computations. Multiple runner files can be initialized to run parallel using `run_DRE.sh` for `DRE_runner.py` and `CDNC_jobscript.sh` for `CDNC_runner.py`. More details on how to run these offline drivers can be found in chapters 4 and 6.

For both *CDNC* and *DRE* runners there are corresponding initializing files that are used to submit runner programs to compute nodes with different parameters in each instance. Output of the *CDNC* runner goes to the `output_CDNC` folder. *ADRE* files are directed to the `output_DRE` folder. Output folders can be altered in code.

Based on the data generated by the data generation tools of this GCM-

forcing analysis tool, the other part (`FI.analysis`) is then used to conduct feature importance analysis. *FI* analysis tools prepare the data for the tools' supervised machine learning (ML) models which are then trained and tested according to their specifications. More in-depth description is given in chapter 5.

The structure of the analysis tool is presented in the figure 1. Green boxes represent folders. Files that need special attention or modifications before using are presented in yellow. The tool operates without the feature importance analysis folder but note that the feature importance analysis needs always the data generation part or then pre-generated data.

For direct effect, currently viable options for features to be interchanged from any AeroCom Phase III CONTROL aerosol model are:

- Mass mixing ratio (*MMR*)
- Relative humidity (*RH*)
- Cloud fraction (*CLT*)
- Refractive index (*n*), including both imaginary and real part

Similarly for indirect effect, the corresponding options are:

- Mass mixing ratio (*MMR*)
- Cloud fraction (*CLT*)
- Cloud liquid water content (*CLW*)
- Updraft (*U*)
- Number mixing ratio (*NMR*)

CAUTION: *NMR* workflow in indirect effect assessment might be faulty and going through the corresponding calculations in code is encouraged upon usage. This problem does not persist in direct effect assessment, only in indirect and in `CDNC_runner.py` pipeline to be specific. The faulty code is included in order to offer a base to build the correct solution on to. The fault is related to how the *NMR* values of the basemodel (ECHAM-SALSA) are scaled instead of being replaced from the comparison model's file. This origins back to when the tool was originally designed there was no size distribution -based data available for other than the basemodel. For more info, see section 4.3.

2 Mandatory Requirements for Operation

In order to be able to run this analysis tool one needs to meet the following requirements:

Overview of the GCMforcing analysis tool layout and tools

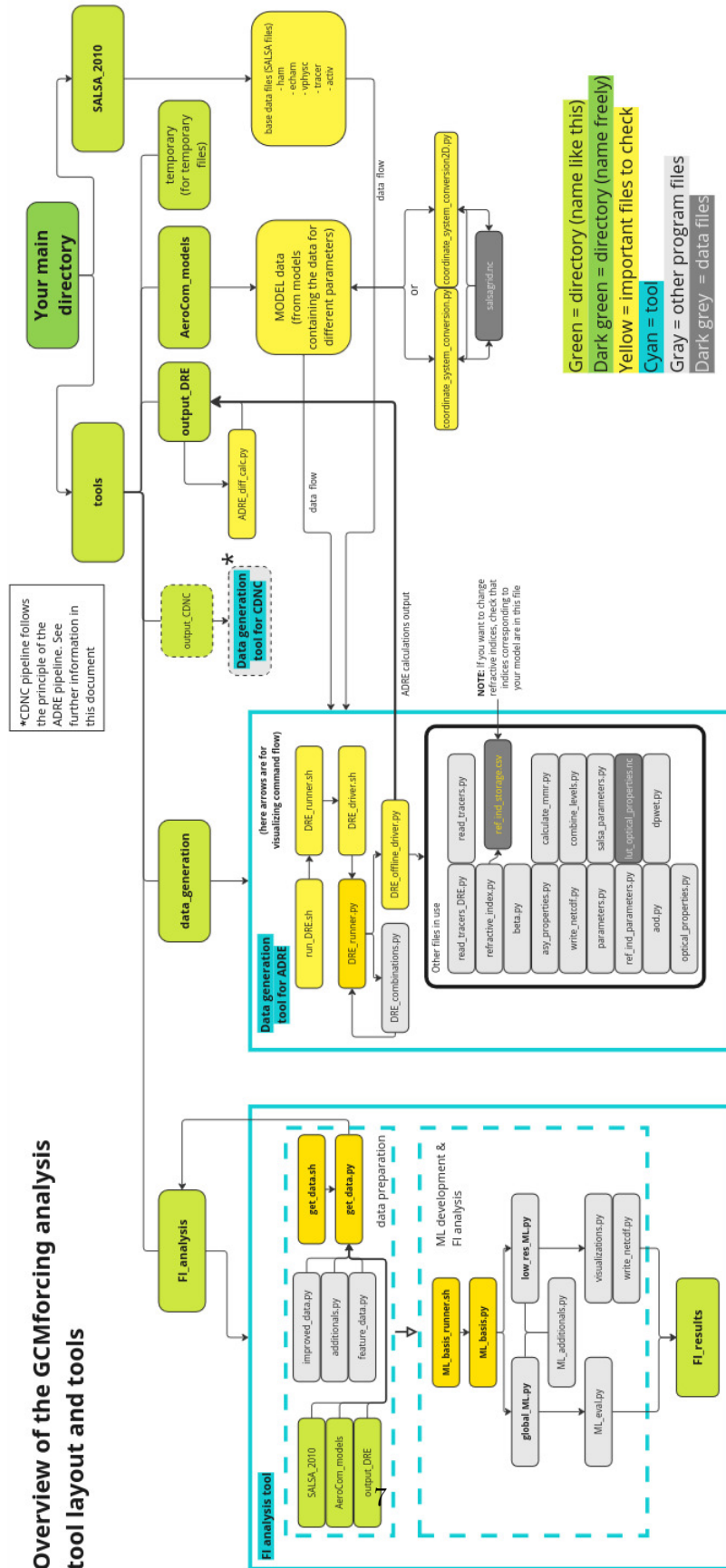


Figure 1: Outlook of the entire analysis tool.

- Latest version of the analysis tool downloaded via <https://github.com/TUT-ISI/GCMforcing>
- Required basemodel (ECHAM-SALSA) files from the study by Irfan et al. (2023) (<https://doi.org/10.5194/acp-24-8489-2024>). (**NOTE:** These files will be publicly available with the next version of this tool but currently they can only be accessed by contacting the author.) The files should have names starting with `vbs_sensitivity_base_2010.01` and also include the following labels:
 - `vphysc`
 - `echam`
 - `ham`
 - `dens`
 - `tracer`
 - `activ`
- Access to AeroCom data server for AeroCom model data. Access must be requested according to instructions in <https://aerocom.met.no/data/download>.
- AeroCom Phase III CONTROL model data from AeroCom server for some feature that is currently possible to analyze with this tool.
- Access to HPC or similar powerful operating system, preferably CSC's Puhti via <https://www.puhti.csc.fi/> (and then also billing units if using Puhti in order to be able to run the programs).

3 Naming the Files

The analysis tool uses files to read in the parameters and model data in order to calculate the asked outcome. The files should be named under specific regulations in order for the program to find them. If files are to be saved in different formats than the ones to be specified below, some changes are required across the system as the analysis tool uses file-reading in multiple occasions.

The files containing simulated data from the ECHAM-SALSA model should be stored under the same folder (see the program chart in the figure 1). This means all the files that the program uses to set up the baseline for calculating the outputs. Files should be named in format

`SALSA2.0_accw_monmean.nc`

or more in general

`{model1}_{specification}_monmean.nc`

where `model1` is the name of the model (e.g. ECHAM6.3-SALSA2.0) and `specification` is the specification label for the file. Note that the `model1` can be arbitrary as long as it matches the files in your SALSA directory. Currently the SALSA files used are named `vbs_sensitivity_base_2010.01`. Note that SALSA files are stored under distinct folder (see the file chart). All the different base data files (specifications) needed are; ham, echam, vphysc, tracer and activ. Some other files may also be needed, but names of these missing files will be presented in the error code. The easiest is to use the files that are currently in the system.

The same applies to the parameter-specific data from the other models. Here the data should be under a distinct folder of their own (see program flowchart). Other model's data should be saved in format

`CAM5-ATRAS_mmrpm1_ModelLevel_2010_monthly_T63L47.nc`

or more in general

`{name}_{parameter}_ModelLevel_2010_monthly_T63L47.nc`

where `name` is the name of the model and `parameter` is same as the one the file contains. Name should also be the same as the one that is given to the models-list in runner-files as well as in feature importance analysis tools. Please do not use ? or ! characters in any file names.

Note that all the files must be resized to be interchangeable. This means that data for every parameter must have the same shape (for time, level, longitude and latitude coordinates). There exists a coordinate system conversion tool in the system files called `coordinate_system_conversion.py` that works for 3D files and `coordinate_system_conversion2D.py` that is for 2D files. Here it is also notable that all model data in AeroCom Phase III CONTROL experiment do not follow the same units so possible modifications may be needed based on your models.

4 Data Generation Tools

Data generation tools are for calculating the labels for each set of features for feature importance analysis. There are separate tools to calculate direct (radiative forcing caused by aerosol-radiation interactions) and indirect radiative effect (radiative forcing caused by aerosol-cloud interactions). Both tools work in similar fashion but the solution for direct effect is more polished and more robustly tested. Please, feel free to make your own modifications according to your needs and taking these following descriptions about the general working principle into account.

4.1 Aerosol Direct Radiative Effect Calculation

See also the step by step instructions to run *ADRE* calculations in 6. This tool is included entirely in `data_generation` directory of the GCMforcing analysis tool repository. Here are the main parts of the tool that contribute to *ADRE* calculations.

4.1.1 DRE_runner.py

The interface of the data generating part of the analysis tool for *ADRE* is in a file titled as `DRE_runner.py`. In `DRE_runner.py`, it is possible to state parameters to be interchanged as well as the models and output directories to be used in the process of calculating *ADRE*, *AOD*, absorption and average backscattering fraction.

`DRE_runner.py` makes automatically different parameter combinations using the given selections. The months are given to it as system arguments from the batch job script (via `run_DRE.sh`) such that one run computes only one month. Program uses Python multiprocessing module for parallelizing parts of the tasks for improved performance. Note that the months to be processed need to be defined in `run_DRE.sh`.

For `DRE_runner.py`, the variables to be checked are listed in table 1. For `DRE_runner.py` to run as it is intended, some modifications to the `run_DRE.sh` and `DRE_runner.sh` may also be needed. In `run_DRE.sh`, number of months under MONTHS-variable can be changed following the formatting of lists in batch job scripts. Additionally, both in `DRE_runner.sh`, as well as in `run_DRE.sh`, the batch job script requests for resources stated in the beginning of each batch job script file can be changed to match the hardware available. Further tuning of these resources might be helpful. See the step by step instructions for *ADRE* calculations in 6 and also documentation of Puhti batch job scripts on <https://docs.csc.fi/computing/running/creating-job-scripts-puhti/>.

4.1.2 ADRE calculation and DRE_offline_driver.py

The program included in `DRE_offline_driver.py` gets its specifications passed on from `DRE_runner.py` and calculates *ADRE* values based on equation 6 in the David J. Delenes and John A. Ogren's article (Delene & Ogren, 2002). The equation in question is represented here in the equation 1. The analysis tool calculates single scattering albedo value backwards from the extinction and absorption files corresponding to the *AOD* file that is used in the calculations. In addition, calculated values are used for the average up-scatter fraction. Outputs are saved to the same folder where the

Table 1: Table of the variables for `DRE_runner.py`

light	The right wavelength for the models (550 nm by default)
modell	The right name such that the program can find the values for the base model
models	The list of the names of the models that are to be used for changing specified parameter fields for the ones of these models
rhsel	Select if the relative humidity field is allowed to be changed to the ones of the models and times specified above when forming the parameter combinations
mmrsl	Same as rhsel but for mass mixing ratio fields
refsel	Same as rhsel but for refractive indices. (Note that the refractive indices are listed in the <code>ref_ind_storage.csv</code> file)
cltsel	Same as rhsel but for cloud fraction
avoid_print_on	Define if the existing files corresponding to specified parameter combinations should be kept or not (If they are kept the iteration for that combination is skipped)
num_cpus	Define the number of CPUs that were defined for single run in <code>DRE_runner.sh</code> batch job script

corresponding *AOD* files are. The direct radiative effect (*ADRE*) can be calculated as mentioned above by

$$ADRE = S_{rad}\phi AOD(1 - C_c)T_{atm}^2(1 - R_s)^2(2R_s\frac{1 - \omega}{(1 - R_s)^2} - \beta\omega) \quad (1)$$

, where S_{rad} is the incident solar radiation, ϕ is the mean daytime value of secant of the solar zenith angle, C_c is fractional cloud amount, R_s is the shortwave reflectance, T_{atm} is the aerosol-free atmospheric transmission, ω is single scattering albedo and β is the up-scatter fraction (Delene & Ogren, 2002). S_{rad} , T_{at} and R_s are set to specified constants based on (Delene & Ogren, 2002).

In order to study the error each change in parameter makes to the *ADRE*, difference between the baseline *ADRE* must be calculated. The difference is called $\Delta ADRE$. See section 6.2 for step by step instructions.

4.2 CDNC and CRE calculation

See also the step by step instructions to run *CRE* calculations in 6. This tool is included entirely in `data_generation_CDNC` directory of the GCMforcing analysis tool repository. Here are the main parts of the tool that contribute to *CRE* calculations.

4.2.1 CDNC_runner.py

CDNC and *CRE* calculations are run by single runner program under `CDNC_runner.py` file. Operation and the logic of the workflow are the same as in the `DRE_runner.py`. The program is commented and the comments instruct what kind of selections should be made. The parameters and paths are to be defined in exactly same way as in 4.1.1 with only difference being the different features in use.

4.2.2 CDNC_offline_driver.py

`CDNC_offline_driver.py` contains the heart of the *CDNC* and *CRE* calculations. All the outputs of the calculations are stored in the output directory according to the built-in naming algorithm that stores files to directories. Directories are defined by the parameter combinations (*MMR*, *NMR*, *CLT*, *U*, *CLW*) that are also used to create the file as well as sub-directories.

`CDNC_offline_driver.py` uses a lot of different files and folders specified in the file's import-commands and in variables in the beginning lines under the running-function. These variables to be verified (at least) are the ones defined in `CDNC_runner.py`.

To date, it is possible to change the outcome of *CDNC* by changing *MMR* and *NMR* fields of SALSA to the ones of the model. Additionally, there is a built in option for updraft changes, but the correctly initialized data is missing so any tests on this feature are yet to be done. In addition to *MMR* and *NMR*, it is also possible to change cloud liquid water content (*CLW*) and cloud fraction (*CLT*) fields to the ones of the other model. This is simply due to the fact that *CDNC* is not affected by *CLT* and *CLW* but *E* is. Therefore there are four (five if updraft data available) parameter fields available to interchange between SALSA and other models.

Energy balance calculations use surface albedo values from SALSA data's echam file to get more accurate local values for forcing. E calculations are based on equations represented in article (Ghan et al., 2013), more specifically equation 25 represented here in the equation 2, where S_0 is the downward solar, f_c is the cloud fraction of lower level clouds, a_s is the surface albedo and a_c is the cloud albedo (Ghan et al., 2013). Other equations of the article are used in some of the interim results.

$$E = 0.25S_0((1 - f_c)(1 - a_s) + \frac{f_c(1 - a_c)(1 - a_s)}{1 - a_c a_s}) \quad (2)$$

4.3 Cautions on the *CDNC* Pipeline and Development Suggestions

***CDNC* and *CRE* calculation pipeline is not fully updated based on the latest improvements made to *ADRE* and *AOD* calculations.**

Here are the main things to consider:

- *CLT* calculation is not up to date. If you wish to use it, you can update it based on the solution in the `DRE_offline_driver.py` where the files with cloud fractions as percentages are converted to fractions. (Currently the GISS and GFDL models are such with this problem).
- Also choosing cloud level based on *CLW* for cloud area fraction dimension reduction from 3D to 2D could be too complex and unnecessary. Here again implementing the solution for *CLT* configuration from `DRE_offline_driver.py` could be beneficial.
- For further development of the indirect radiative effect calculations in general, it is recommended to follow the results in eg. <https://agupubs.onlinelibrary.wiley.com/doi/10.1002/jgrd.50567>.

5 Feature Importance Analysis Tools

5.1 In General

The feature importance (*FI*) analysis part of the system should be located under entirely own directory that should be located according to the program chart in the figure 1. The feature importance analysis includes two different solutions for studying feature importances of the aerosol direct (and with minor modifications also indirect) radiative effects. The tool fetches data using program made for searching and prepares data ready for the actual feature importance calculations. The tool then develops machine learning models based on the set-up and maps the feature importance values based on these models into a netCDF file while forming also other desired statistics and plots.

The data can be distributed to the ML models in two different ways. First option is to flatten all the offered data and feed it to a single ML model that then develops *FI* values and metrics. Here the (normalized) coordinates are therefore added as features to allow ML model to get some sense of the effect the spatial variability has on the uncertainty. The other option is to develop multiple models to smaller domains (with maximum resolution of the coordinate grid used in the study). This way the spatial variability of the data is not concerning ML models as they should only learn the effects happening in their own domain.

5.2 The Initialization of *FI* Analysis

The data generated by GCMforcing data generation tools must be prepared for *FI* analysis. The data is prepared using `get_data.py` that then fetches, reshapes and stores the feature and label data to parquet files ready to be

used for ML models. Here again, specifying the paths, models and features to be used are up to the user.

The pipeline of the data preparation the `get_data.py` initializes is relatively long and complex. Essentially what happens is that `get_data.py` requests `improved_data.py` to search all the relevant feature (AeroCom) and label (the calculated *ADRE/CRE*) files from the directories of this analysis tool. `improved_data.py` uses `feature_data.py` and `additional.py` to aid the data handling as eg. *RH* and *MMR* data must be flatten to 2D by calculating the respective burdens in order for the data to match the dimension of the label data. For step by step instructions for data generation visit 6.

After the data is initialized, it can be used in `ML_basis.py` which is the operating interface of the actual *FI* analysis tool. In `ML_basis.py` it is possible to specify the scale (linear or logarithmic), ML model, domain size and whether or not to use domains as well as output folder and cluster configuration (for Dask parallelization). `ML_basis.py` then initializes the data further by either dividing it to different domains (domain-wise models) or adding coordinates as features (all data -models). `ML_basis.py` is run by `ML_basis_runner.sh` in which it is important to remember to specify the resources according to the chosen ML model and data split. Simple linear regression (LR) with all data -type of training only needs less than 5 minutes of running time on 6 GB of memory where as Random Forest (RF) developed on domain-wise data can take up to 9 hours with 6 CPUs and over 60 GB of memory (estimated on Puhti).

5.3 The Domain-wise ML Development

The domain-wise models are initialized as discussed above and return the following outputs; map of the most important features based on *FI* values both in png:s and in netCDF formats, R^2 map in png and netCDF, *RMSE* map in png and netCDF and metadata for the model including combined *FI* values from all the grid boxes as well as global averages of R^2 and *RMSE*. The domain-wise models use `Lower_resolution_ML.py` and `visualizations.py` for the results. For step by step instructions on the domain-wise model operation, please check the section 6.

5.4 The All Data ML Development

The all data -models are initialized as discussed above and return the following outputs; bar plot of the most important features based on *FI* values in png-format, R^2 and *RMSE* stored in txt-file metadata, prediction error plot in png, learning curve in png and partial dependence plot in png. All data -models use `global_ML.py` and `ML_eval.py` for its results. For step by step instructions on the all data -model operation, please check the section

6.

5.5 Other Files in *FI* Analysis Tools

5.5.1 `additional.py`

The `additional.py` file contains subroutines for data fetching programs. Modifications to this file are only needed if alternations to data handling are wanted to be made or if new features are introduced and reading the data from those features needs subroutines.

5.5.2 `ML_additional.py`

The `ML_additional.py` file contains subroutines for ML initialization programs like adding the coordinates to features and dividing the data to domains. Modifications to this file are only needed in relatively special cases but adding new ML model is one of them. It can almost be said in other words that if you think these functions need modifications, then you also know what to do.

6 Step-by-Step Instructions

6.1 Calculating *ADRE*, *AOD*, Absorption and Average Backscatter Fraction Values on Compute Nodes

1. Ensure that your files are organized as stated in the figure 1 and that you meet the requirements stated in section 2.
2. Open `DRE_runner.py` and select the models and variables that are allowed to be interchanged when calculating output values. For example if *MMR* is set to 'mmr', *MMR* field can be fetched from the model data. Combinations are formed based on these permissions, so at maximum with all the current parameters made available, max amount of combinations is 16.
3. Choose if you want to avoid reiterating those iterations that already have corresponding output files by selecting `avoid_print_on=True`.
4. Check that the number of CPUs match the number that is available for one job (cpu-per-task).
5. Check that the paths are set correctly.
6. Open `DRE_runner.sh` and ensure that the resources requested in the SBATCH commands are available. Current memory and CPU alloca-

tions shouldn't be changed unless better configurations are found by testing the system. Names for logging files and emailing commands can be changed.

7. Open `run_DRE.sh` file and ensure that all the months wanted are listed in the `MONTHS` variable. This batch job script is **ONLY** used for submitting `MONTHS` length amount of jobs that are then initialized according to `DRE_runner.sh` file and the month given to them. Therefore the requests for this job are very light.
8. Head to the terminal and navigate to the folder that contains the `run_DRE.sh` file and submit the job to the queue by command `sbatch run_DRE.sh`.
9. View the job error and job output files in the working directory.
10. Output files should now be in the output folder each under respective subdirectories.

6.2 Calculating $\Delta ADRE$

- Follow the steps in the step-by-step guide above for *ADRE*-values.
- Run the `ADRE_diff_calc.py` and the values are stored in the respective directories the comparison files are fetched from.
- Follow the output log and from that and/or from the output folders see that all the $\Delta ADRE$ values are calculated.

6.3 Calculating *CDNC* and (aerosol-cloud interaction based) Energy Balance on Compute Nodes

1. Ensure the files are organized as stated in the figure 1 and that you meet the requirements stated in section 2.
2. Open `CDNC_runner.py` file and select all the models and other calculation settings based on instructions in the comments. Define the paths.
3. Select the parameters that are allowed to be changed. Logic selecting them is the same as in the *ADRE* case.
4. Remember to match the number of CPU's with the resources available.
5. Define resources for single iteration in the `CDNC_runner.sh` file. Generally, you can use the same values as for `DRE_runner.sh` but note that

these calculations require more memory. Remember to also adjust the time based on previous runs.

6. Ensure that the batch job script in `CDNC_jobscript.sh` is defined right.
7. Head to the terminal and navigate to the folder that contains the `CDNC_jobscript.sh` file and submit the job to the queue by command `sbatch CDNC_jobscript.sh`.
8. Monitor the job error and output files.
9. Check the `CDNC_output` folder to see that the files were formed correctly.

6.4 Initializing data for *FI* analysis using `get_data.py`

1. Open `get_data.py` and ensure all parameters are set correctly and all the paths and data exists. If you have no data for the labels, please see the previous two step-by-step instructions.
2. Ensure the files are named and organized as stated in the Naming the Files -section and in figure 1.
3. Ensure that all the models and all the features that are wanted to be used are listed in the program.
4. If you have some other features than *RH*, *MMR*, *REFIND* or *CLT*, go check 6.8 "Adding New Features".
5. Choose if you want to use *ADRE* or $\Delta ADRE$ in your *FI* analysis.
6. Head to the `get_data.sh` and specify the system requirements in the script based on your system and knowledge. Choose a prefix for your output.
7. `sbatch` the `get_data.sh` in your command prompt after navigating to the correct directory.
8. Ensure that you get the desired parquet files in your directory.

6.5 Calculating Feature Importances on Domain-vise ML Models

1. Ensure that you have preprocessed data for ML stored in parquet files under some common prefix. If not, check 6.4.
2. Open the `ML_basis.py` file.

3. Choose the prefix which the parquet files you just made with `get.data.py` is stored with.
4. Choose if ten based logarithm values of labels are wanted to be used as the target values.
5. Choose which ML model you want to use. Currently the options are LR for linear regression, RF for random forest, DT for decision tree and GB for gradient boosting. For more ML models, check 6.9.
6. Choose to use multimodel.
7. Define the domain dimensions for lowered resolution. For example if 2-by-2 is used, the data will be divided so that data inside each domain size of 2-by-2 grid boxes will be used for that domain and the resolution of the map will therefore be lowered as now instead of individual grid boxes, the map is represented by domains the size of 2-by-2 grid boxes.
8. Check that the naming of the output files are correct.
9. Use the `ML_basis_runner.sh` to allocate correct amount of resources to this task (Try with smaller requests and enlarge when needed. Monitor this with job feedback system.) and submit the job to the queue from the terminal after navigating to the `FI_analysis` folder.
10. Monitor the tool via the error and output files
11. Ensure that the all the desired output is in the output folder.

6.6 Calculating Feature Importances on All Data ML Models

1. Follow the steps 1 to 5 in 6.5.
2. Choose not to use multimodel.
3. Check that the naming of the output files are correct.
4. Use the `ML_basis_runner.sh` to allocate correct amount of resources to this task (Try with smaller requests and enlarge when needed. Monitor this with job feedback system.) and submit the job to the queue from the terminal after navigating to the `FI_analysis` folder.
5. Monitor the tool via the error and output files
6. Ensure that the all the desired output is in the output folder.

6.7 Adding New Aerosol Models to the System

1. Ensure the files are named and organized as stated in section 3 and in the figure 1.
2. Ensure that the files of this new model match the dimensions of the basemodel. If not, transform them using the `coordinate_system_conversion.py` file in the `data_generation` directory.
3. Open the runner file you are planning to use and add the name of the model to the model list.
4. Add data of the refractive indices of that model to the table in `ref_ind_storage.csv` file under the other models data, if refractive indices data of these models are planned to be used at all.
5. Follow the instructions for that runner and get the target data for that model.
6. Add the name of that model to the list of the names of the models in the *FI* analysis tool you are planning to use.
7. If there are exceptions for this model for any of the features it has, remember to update necessary exceptions also to the corresponding parts of the program. Look for reference in the code where there might be similar solutions made for existing models.
8. Run the *FI* analysis tool.

6.8 Adding New Features to the System

1. Ensure the files are named and organized as stated in section 3 and in the figure 1. Also make sure that the feature you want to add exists for the models you plan to use in the first place.
2. Ensure that the files of this new feature match the dimensions of the basemodel. If not, transform them using the `coordinate_system_conversion.py` file.
3. If the new feature affects *ADRE*, *AOD*, *CDNC* or *CRE*, make the possible modifications to the codes in the corresponding offline drivers and/or to the additional functions according to the effect the feature has to the calculations. For example, if you want hygroscopicity from different aerosol models, you would have to modify at least offline river to load the data and to pass it to `dpwet.py` that should then be altered to take that data into account.

4. Modify the file naming of that driver to include the abbreviation of this feature. Note that this has to be done also for the folder naming. Changes to parameter combination formation must be also made. Again, seek example from other features.
5. Additionally, remember to select month of data that is used for *CDNC* / *AOD* calculations with offline drivers' `cdo_copy` function and `selmon` variable. Seek examples from other similar parts in the code.
6. Open the runner file you are planning to use and add the name of the feature to the feature list.
7. Follow the instructions for that runner and get the target data.
8. Add the name of the feature to the list of the names of the features in `get_data.py`.
9. Add the feature to the searching algorithm in the data generating files of `improved_data.py` and `feature_data.py`. Seek example from other features already implemented in system. Adding additional debugging statements may be helpful.
10. Create additional subroutines, if needed, inside the `additional.py` file and import them to the other files if needed. It is again encouraged to mimic the workflow of the other features and adapt those to the new features.
11. Note: Program might still raise errors but they should be relatively easy to locate and fix based on the error messages.
12. Run the *FI* analysis tool and take notes on which feature is represented by which number (modify the metadata output file and other parts if needed).

6.9 Adding New ML Models

1. Add the abbreviation of this new ML model to the `ML_basis.py` following the example set by other models.
2. Import the required libraries to the file you are planning to implement this ML technique in. It is good practice to implement the model to both all data and domain-wise cases.
3. Add the new ML model to be another option following the example set by current ML models. It is again good practice to implement parameter tuning, possibly using the same method other ML models are using.

4. Go through the entire pipeline of the FI value calculations and ensure that everything is set
5. Run and test the newly implemented model and take notes on the performance to be able to set the requirements in `ML_basis_runner.py` accordingly.

7 More Information

For more information on Puhti operation, visit for example <https://docs.sc.fh.fi/>. For more information on the parameter tuning, evaluation metrics, ML model workflows and working principles, visit <https://scikit-learn.org/stable/>. scikit-learn has also tutorials for other ML models not currently implemented so referring those could be helpful. See AeroCom website <https://aerocom.met.no/> for documentation on their models and how to access their data. Good common sources for information related to calculations used in this tool are: *Atmospheric Chemistry and Physics* by Seinfeld & Pandis (ISBN: 978-1-118-94740-1), article by Delene and Ogren (Delene & Ogren, 2002) for direct effect calculations, article by Ghan et al. (Ghan et al., 2013) for indirect effect calculations and article by Rengasamy et al. (Rengasamy et al., 2022) for feature importances.

References

- Delene, D. J., & Ogren, J. A. (2002). Variability of Aerosol Optical Properties at Four North American Surface Monitoring Sites. *Journal of the Atmospheric Sciences*, 59(6), 1135–1150. [https://doi.org/10.1175/1520-0469\(2002\)059<1135:VOAOPA>2.0.CO;2](https://doi.org/10.1175/1520-0469(2002)059<1135:VOAOPA>2.0.CO;2)
- Ghan, S. J., Smith, S. J., Wang, M., Zhang, K., Pringle, K., Carslaw, K., Pierce, J., Bauer, S., & Adams, P. (2013). A simple model of global aerosol indirect effects. *Journal of Geophysical Research: Atmospheres*, 118(12), 6688–6707. <https://doi.org/10.1002/jgrd.50567>
- Rengasamy, D., Mase, J. M., Kumar, A., Rothwell, B., Torres, M. T., Alexander, M. R., Winkler, D. A., & Figueredo, G. P. (2022). Feature importance in machine learning models: A fuzzy information fusion approach. *Neurocomputing*, 511, 163–174. <https://doi.org/10.1016/j.neucom.2022.09.053>